Electra Central AC — IR Protocol

Reverse-engineered command protocol for a central (ducted) Electra-family air conditioner, in Broadlink raw-IR (`b64`) form.

Rev 2026-07-21 · model unknown · github.com/danielrosehill/electra-central-ac-ir-codes

At a glance
Manchester bi-phase line coding · 34-bit core payload sent 3× · **no checksum**. Power is a single **toggle**; separate state frames carry mode, fan and temperature. Because there is no checksum, every command can be **generated** — the full 120-code matrix was synthesized from 7 captured frames.

1 Hardware & capture

The signal was captured with a **Broadlink RM4C mini** IR transceiver in learning mode (`python-broadlink` `enter_learning()` / `check_data()`), pointed at by the AC's handheld remote. All codes in this repository are in the Broadlink raw-IR container: a base64 string beginning `Jg...` (`0x26`), directly usable with `python-broadlink` `send_data()` or Home Assistant `remote.send_command` (`command: "b64:…"`).

Broadlink encodes durations in **ticks** of ≈ 32.84 µs. Throughout this document a duration is given in ticks.

2 Physical layer

2.1 Line coding

The frame is **Manchester bi-phase**, not simple pulse-distance. The tell: a histogram of a capture shows both marks **and** spaces clustering at integer multiples of a single base unit.

$$\text{half-bit } T_{\text{h}} \approx 31.7 \text{ ticks} \quad (\approx 1.04 \text{ ms})$$

Each core bit spans two half-bits (one transition):

Bit value	Half-bit levels	Meaning
`0`	mark → space `(1,0)`	carrier on then off across the bit period
`1`	space → mark `(0,1)`	carrier off then on across the bit period

Consecutive equal half-bits merge into one longer mark/space, which is why raw captures show runs of 1×, 2× and (at boundaries) 3× the base unit.

2.2 Frame structure

```
[ leader ] [ 34-bit core ] [ 34-bit core ] [ 34-bit core ] [ lead-out gap ]
```

- **Leader** — a short mark/space pair (≈ 3 half-bits each) preceding the payload.
- **Core** — the 34-bit payload below, transmitted **three times** back-to-back for reliability (all three copies identical).
- **Lead-out** — a long gap (≈ 3333 ticks) closing the transmission.

3 The 34-bit core

Bit 0 is the first core bit. Fields occupy fixed positions; every other bit is constant. power mode fan temp fixed



core bit index (0–33)

Bits	Field	Encoding
0	power flag	`1` = this is the **power-toggle** command · `0` = a state / settings frame
2–3	mode	`cool = 01`, `heat = 10` (MSB = bit 2). Other modes untested.
4–5	fan	`low = 00`, `med = 01`, `high = 10`, `auto = 11` (MSB = bit 4)
11–14	temperature	4-bit value = **°C − 15**, MSB = bit 11 (`16 °C → 0001` … `30 °C → 1111`)
others	fixed	Constant framing bits; identical across all captured frames.

Power is a toggle — read this
One code flips the unit on ↔ off (the "on" and "off" captures are byte-identical). **State frames do not power the unit on** — sending `cool / 22 / high` to an off unit does nothing. To control it: send the power toggle to switch on, *then* send state frames for mode/temp/fan.

4 Worked example

The captured frame `cool / high / 20 °C` decodes to this core (bit 0 first):

```
00011000000010100000000000000000010
```

bit 0	`0`	→ a state frame (not the power toggle)
bits 2–3	`01`	→ **cool**
bits 4–5	`10`	→ **high**
bits 11–14	`0101` = 5	→ 5 + 15 = **20 °C**

5 Generating any command

With no checksum, a new command is built by rewriting only its field bits in a template frame and re-encoding to Manchester:

$$\text{temp bits}(11..14) = \text{bin}(°\text{C} - 15), \quad \text{fan bits}(4,5) = \text{code}_{\text{fan}}, \quad \text{mode bits}(2,3) = \text{code}_{\text{mode}}$$

Generated frames were validated two ways: (a) **in-silico** — a generated frame reproduces the corresponding real capture to within IR jitter (≤ 4 ticks); and (b) **on the physical AC** — a generated `cool / 25 °C` command drove the unit correctly.

Reproduce it yourself

```
python electra_ir.py decode raw_captures/anchor_20.b64   # -> fields
python electra_ir.py gen cool 22 high                    # -> Broadlink b64
python electra_ir.py matrix > codes.json                 # rebuild all 120
```

6 Methodology (how the map was found)

1. **Capture** a handful of states from the physical remote in Broadlink learn mode.
2. **Decode** the raw pulses; the mark/space multiples-of-one-unit histogram reveals Manchester coding → a stable 34-bit core repeated 3×.
3. **Diff** captures: −1 °C vs +1 °C changed one small region → temperature is a localized field. Two **known** temperatures (20 and 24) pinned °C − 15, and crucially **no other bit moved** → there is no checksum.
4. **Generate & verify** — synthesize codes, validate in-silico and on hardware.
5. **Repeat** for fan (bits 4–5) and mode (bits 2–3), each an isolated field.

Result: 7 real captures were enough to generate and cross-check a 120-code matrix (cool + heat × 16–30 °C × 4 fan speeds).

Codes provided as-is; verify against your own unit. Model number still unknown — contributions (model, extra modes, swing, other remote variants) welcome via PR. MIT-licensed.